

What a cheap spectral probe can and cannot tell you about
which solver to use:
a leave-one-family-out evaluation with negative results

Tricha Anjali

Raghu Venkat

Numberz.ai Inc., Alpharetta, GA

14 September 2026

Abstract

Automated solver selection is usually reported as a classification accuracy on a pool of matrices split at random. That number tells an engineer very little about what happens when a problem family she has never solved before arrives on a Tuesday. We evaluate a selector built on a cheap spectral probe, thirty Lanczos steps on the Jacobi-preconditioned operator, choosing among six classical methods on seventy-two generated symmetric positive definite operators drawn from six families with swept parameters. We hold out whole families rather than individual matrices, and we score in regret against oracle selection rather than in accuracy, because picking a method one per cent slower is not the failure that picking one eighty per cent slower is. The tail and accuracy criteria fail in every environment we have run: ninetieth-percentile regret is 0.35 against a registered bar of 0.20, and winner accuracy between 0.400 and 0.492 against 0.80. The median criterion depends on the numeric stack, missing its bar of 0.05 under three of the five library combinations we tried and meeting it under two, while every other quantity is identical across all five. Those percentiles are conditional on the chosen method converging, which is what a percentile over a population containing failures silently computes; six of the sixty-five choices did not converge, and read unconditionally the same tail is 10.66. We report both everywhere, because the gap between them is the paper's subject. The catastrophes are concentrated and the shortfall is not: one family of six carries five of the six choices that failed to reach the tolerance within the declared iteration cap, and removing it drops those from six to one, while the registered median bar is missed on four folds of six and the tail bar on four. In that family the right answer flips from *almost anything works* to *only one method works*, and nothing in the training families contains that flip. Four further results are negative and, we argue, more useful than the positive ones. A distance-based abstention rule calibrated inside the training families refuses seventy-eight per cent of held-out operators, moves the tail it was built to protect only from 0.352 to 0.347, and doubles the number of operators on which the deployed choice never converges, from six to twelve. The Ritz condition estimate alone beats the full twelve-feature selector at the median, 0.00 against 0.10, and is catastrophic in the tail, 2.18 against 0.35: it fails our registered baseline-to-beat test at the median, and the other eleven features earn their place only in the tail. On twenty-three of sixty-five operators the fastest method is unpreconditioned conjugate gradients, so a selector restricted to preconditioners would be wrong on a third of this grid by construction. And on three of sixty-five operators the probe costs more than the whole fastest solve it is meant to inform, by up to a factor of seven; charging that probe to the selector on every operator, as a deployment must, moves median regret from 0.10 to 0.41 and the tail from 0.35 to 3.40. We also report that on twenty-five of four hundred and thirty-two runs the recurrence residual carried inside conjugate gradients

announced a convergence that the true residual did not support, so a harness trusting it would have recorded speedups that did not happen. The pre-registration, the generators, the solvers and the hashed result files accompany the paper, and we say which experiments did not run.

1 Introduction

A solver-selection result is easy to report and hard to use. The convention is a best-method classification accuracy over a pool of sparse matrices, split at random into training and test sets. Lighthouse [1] applies machine-learned classification over a solver taxonomy to choose among PETSc and Trilinos solvers, and MM-AutoSolver [2] selects an iterative solver and a preconditioner together from multimodal features, reporting predictive accuracy and speedup; both report what they measured, and neither reports accuracy alone. Outside numerical software the algorithm-selection literature has already formalised much of what we measure: ASlib [3] defines feature-computation cost as part of a selection system’s performance and scores against a virtual best solver, which is our oracle under another name. An engineering organisation deciding whether to put such a selector in front of its simulation pipeline needs three other things: what happens on a problem family the selector has never seen, what a wrong choice costs, and whether the selector knows when to decline.

This paper reports those three things for one simple selector. It carries no learned solver and no discovered method. Every candidate is a classical preconditioner that has been in the literature for decades, every constant is declared in advance, and the operators are generated from stated discretisations by the accompanying script, so the study reproduces with no data download. That simplicity is the point. When the selector fails we can name the quantity responsible without pointing at the opacity of a fitted function, and most of the useful results below are failures. We are not proposing a new framework for algorithm selection. We are reporting a controlled case study in which conditional runtime summaries, selection failures, feature-acquisition cost and residual verification lead to four materially different judgements of the same system.

We make six contributions. First, an evaluation protocol that holds out whole operator families rather than individual matrices, which is the deployment- realistic case and which our own selector fails. Second, a regret-based score against oracle selection, reported at the median and the ninetieth percentile, alongside the accuracy the literature reports, so that the two can be compared on the same runs. Third, a measured account of what a distance-based abstention rule does when a new family arrives, which is refuse nearly everything and help nothing. Fourth, the observation that on nearly a third of this grid the fastest method is no preconditioner at all, so a benchmark that requires the system to choose a preconditioner can manufacture an improvement by excluding *use none* from the action space. Fifth, the observation that the cheap probe is a net loss on cheap problems, with the crossover located. We add a second regret that charges it; charging feature cost is established practice [3] and we claim only the measurement. Sixth, a measurement of how often the residual recurrence inside conjugate gradients misreports convergence on this grid. The gap between the recurrence and the true residual is classical numerical analysis [4, 5] and is not our discovery; the contribution is the rate on this benchmark and its consequence for selection, because a selector is only as good as the oracle labels it is scored against and a harness that trusts the recurrence will label some unconverged runs as winners.

2 What was fixed before the first solve

The protocol was written and hashed before any operator in the declared grid was scored, and is reproduced in `PREREGISTRATION.md` with its SHA-256. It fixes the six families and their swept parameters, the six methods, the probe budget, the twelve features, the learner and its hyperparameters, the leave-one-family-out partition, the regret definition, the abstention rule and the success criteria. One addendum was recorded, dated and hashed before the sweep ran, after implementation validation on two of the seventy-two cells: it defines convergence on the true residual rather than the recurrence residual, and it states how failures and the oracle interact. It changes no parameter of the grid, the methods, the features, the partition or the criteria. Section 8.8 reports the measurement that made the first half of the addendum necessary. A second addendum was recorded *after* the results were read and is labelled as post-hoc in the file: it adds the deployment regret of §8.7 beside the registered score, and changes no parameter and no criterion. Every criterion in this paper is scored on the registered regret and on nothing else.

The pre-registration states, before any result was read, that we expected the primary criterion to fail on at least one fold. It did.

3 Operators

Six families, each with a swept parameter, because the case this study is about is an operator that moves while the problem stays the same. F1 is two-dimensional heterogeneous conduction on a five-point cell-centred stencil with harmonic face conductivities and a smoothed log-normal conductivity field, swept over decadic contrast. F2 is the three-dimensional seven-point version of the same. F3 is two-dimensional anisotropic diffusion, swept over the anisotropy ratio. F4 is two-dimensional conduction with piecewise-constant conductivity on twelve random circular inclusions, swept over the jump magnitude. F5 is the graph Laplacian of a random geometric graph in the plane, shifted by σI to make it positive definite, swept over σ . F6 is a shifted stiffness $K + cM$ with a lumped mass, swept over c . Three sizes per family and four sweep values give seventy-two operators between 1.4×10^4 and 1.2×10^5 unknowns. The right-hand side is a fixed pseudo-random vector seeded from the operator identifier and is the same for every method.

4 The methods and the harness

Six candidates, all classical: M0 unpreconditioned conjugate gradients, M1 Jacobi, M2 SSOR at $\omega = 1$, M3 incomplete Cholesky with zero fill, M4 a Neumann polynomial preconditioner of degree four, and M5 a Chebyshev polynomial preconditioner of degree eight whose interval comes from the probe. M5 is the only method that consumes the probe, and it is charged for it.

All six run through one preconditioned conjugate gradient implementation so that iteration counts and operator applications are comparable. The tolerance is a relative residual of 10^{-8} , the iteration cap is 20,000 and a single solve is additionally capped at thirty seconds of wall clock. Convergence is decided on the true residual, recomputed every hundred iterations and at exit. A method that does not converge is recorded as a failure and is never quietly dropped.

5 How the probe works

Thirty Lanczos steps with full reorthogonalisation on the symmetrically scaled Jacobi-preconditioned operator, from a seeded random start, returning the Ritz values of the tridiagonal. From those: the log Ritz condition estimate, the log extremes, the three Ritz quartiles normalised by the largest, the largest relative gap between consecutive Ritz values, and the mean and standard deviation of the log Ritz spectrum. From one pass over the matrix: $\log n$, nonzeros per row, and the minimum diagonal dominance ratio. **Twelve features, fixed in advance.** The pre-registration enumerates these twelve and then miscounts its own list as eleven; `FEATURE_NAMES` has carried these twelve in this order since before the sweep. The enumeration is authoritative and the word was wrong. No feature was added after a result was read. The correction is recorded as addendum 3 and the locked hash can be checked against the enumeration.

Three of the twelve are linearly dependent by construction: the log condition estimate is the difference of the log largest and log smallest Ritz values, with a measured residual of 4.4×10^{-16} . We keep all three because the set was fixed in advance, and we report in §8.5 what that dependence does to the abstention rule that consumes it.

6 The selector in full

So that the results can be read without inferring the implementation: one gradient-boosted regressor per candidate method, two hundred trees, depth three, learning rate 0.05, seed 0, untuned. Each regressor predicts $\log_{10}$ of end-to-end time for its own method, and the selector takes the argmin of the predictions over the candidate set. Runs that did not converge carry no time and are dropped from that method’s training rows rather than encoded as a large value, so a method that fails often is fitted on fewer points and is never penalised for the failures; this is a limitation and we chose it, because any encoding of failure as a number is a free parameter we did not want to choose after seeing results. Within each fold the training partition is the five other families; a seeded twenty-five per cent of it is held aside as the calibration split and is used only for the abstention threshold and its covariance. The regressors never see it. No operator of the held-out family enters any fit, scaling, covariance or threshold. The declared fallback on a refusal is the method with the best median rank on that fold’s fitting partition, which is Jacobi on five folds and no preconditioner on one; when the fallback runs it is charged the probe that was already paid to decide to refuse.

7 How regret is defined

For an operator A , with $\hat{m}$ the selector’s choice and m_o the fastest method that converged, regret is

$$\rho(A) = \frac{T(\hat{m}, A) - T(m_o, A)}{T(m_o, A)},$$

the excess end-to-end time as a fraction of the oracle’s, where T includes setup and, for M5, the probe. A choice that does not converge has infinite regret and is counted separately. Seven of the seventy-two operators had no converging method at all; they carry no selection signal and are excluded from the statistics and reported as a count. Six of those seven are the high-contrast cells of F1, which is why that fold carries six test operators in Table 3 where the others carry eleven or twelve. Sixty-five operators remain.

8 Results

8.1 The registered criteria, and how they fell

	median ρ	$p_{90} \rho$	p_{90} uncond.	did not converge
Selector, twelve features	0.10	0.35	10.66	6 of 65
Ritz condition estimate alone	0.00	2.18	inf	8 of 65
Null control, shuffled labels	0.14	1.28	2.89	1 of 65
Selector with abstention and fallback	0.00	0.35	inf	12 of 65
Selector, probe charged (post-hoc)	0.41	3.40	10.78	6 of 65

Table 1: Pooled over the six held-out families, $n = 65$ in every row. The first two columns are **conditional on the chosen method converging**, over the denominator 65 minus the last column; they are what a percentile taken over a vector containing infinities computes, and the registered bars, a median of 0.05 and a ninetieth percentile of 0.20, were written in those terms and are scored on the first row. The third column keeps the failures at infinity and is taken by nearest rank, so a population failing on more than a tenth of its operators reports *inf* rather than a finite number that excluded them. The last row is the post-hoc accounting of §8.7.

The tail criterion and the accuracy criterion fail in every environment we have run. The median criterion depends on the machine. Table 2 gives the same `score.py` on the same `raw.jsonl` under five numeric stacks: the median misses its bar of 0.05 in three of them and meets it in two, and winner accuracy moves between 0.400 and 0.492 against a bar of 0.80. Every other quantity in this paper is identical across all five, including the ninetieth percentile at 0.3519, the six non-converging picks, the coverage of 0.2154, the Ritz-only row and the abstention row.

The mechanism is visible in which statistics move. Median and accuracy are decided by an argmin over predicted times, and on many operators two candidates are separated by less than the arithmetic noise between library versions, so the argmin flips. The tail is decided by operators where one method fails and another does not, and no rounding changes that. A statistic that moves with a library version is measuring the tie, and the frequency of near-ties is itself a result: on this grid, *which method is fastest* is often not a well-posed question, while *which methods finish* always is.

We report the median as a range across environments and we pin the stack in the ancillary files. Numbers quoted elsewhere in this paper are from the first row of Table 2 unless stated, and the conclusions do not turn on which row is used.

Read on the first row, the median is 0.10 against 0.05, the ninetieth percentile is 0.35 against 0.20, and winner accuracy is 0.415 against 0.80. The separation from the null control is worth reading closely, because it is where the evidence of signal actually lives. At the median the two are close, 0.10 against 0.14, and nothing should be concluded from that gap. In the tail they are far apart, 0.35 against 1.28, and the null control’s single non-converging pick against the selector’s six is the one column where the null looks better. The probe carries selection information, and it carries it in the shape of the distribution.

The registered baseline-to-beat is the Ritz condition estimate alone, and section 9.3 of the pre-registration obliges us to say so if the twelve-feature selector does not beat it on median regret. It does not: 0.10 against 0.00. The ten additional features pay for themselves only in the

numpy	scipy	scikit-learn	median ρ	accuracy
2.2.6	1.15.3	1.7.2	0.103	0.415
2.4.6	1.17.1	1.7.2	0.117	0.400
2.4.6	1.17.1	1.6.1	0.117	0.400
2.4.4	1.17.1	1.8.0	0.000	0.492
2.4.6	1.17.1	1.5.2	0.000	0.492

Table 2: The same scoring script on the same raw results under five numeric stacks. Only the median and the accuracy move. The registered median bar is 0.05 and the registered accuracy bar is 0.80. Pinning scikit-learn alone is not enough: rows one and two share a scikit-learn version and disagree.

tail, where they take the ninetieth percentile from 2.18 to 0.35.

8.2 Fold by fold

Held-out family	n	median ρ	$p_{90} \rho$	p_{90} unc.	no conv.	accuracy	coverage
F1 conduction 2D	6	0.00	0.00	0.00	0	1.00	0.00
F2 conduction 3D	11	0.00	0.33	3.32	1	0.82	0.00
F3 anisotropic	12	0.14	0.34	0.35	0	0.17	0.25
F4 inclusions	12	5.57	7.96	inf	5	0.25	0.33
F5 graph Laplacian	12	0.13	0.19	0.19	0	0.17	0.00
F6 shifted stiffness	12	0.11	0.27	0.27	0	0.42	0.58

Table 3: Per fold, with the same two readings as Table 1. Bold marks a cell that misses its registered bar. F4 carries five of the six non-converging picks and is the only fold whose unconditional tail is infinite, but it is not the only fold that misses: the median bar of 0.05 is missed on F3, F4, F5 and F6, and the tail bar of 0.20 on F2, F3, F4 and F6. F1’s perfect column rests on six operators, half its family having been excluded because no candidate converged on them.

Two things are true at once and the second is easy to lose. Non-converging choices are concentrated almost entirely in F4: removing it takes them from six to one, and it is the only fold whose unconditional tail is infinite. But the registered bars are missed much more widely than that. Four folds miss the median bar of 0.05, and four miss the tail bar of 0.20. The *catastrophes* are one family and the *shortfall* is most of the grid.

F4 is the family where the answer changes character. At a jump magnitude of 10^6 and the largest size, five of the six methods fail outright and only the Chebyshev polynomial preconditioner converges; at the smaller jumps on the same family every preconditioned method converges within a factor of four of the best. Nothing in the five training families contains that flip, so the selector extrapolates from a regime where the choice barely matters into one where it decides whether the solve happens at all. The Ritz condition estimate alone is worse still on this family, at a conditional ninetieth-percentile regret of 11.34 and an unconditional one of infinity.

What this fold establishes needs stating narrowly. It establishes that *this* feature set with *this* learner, trained on the other five families, chose badly on F4. It does not establish that the

spectral representation cannot support selection on F4, nor that training coverage or a different learner would not help. The experiment does not separate those three causes.

8.3 Why accuracy is the wrong number

The selector picks the oracle on 27 of 65 operators, so 38 choices are misses. Six of those never reach the tolerance, leaving 32 that are suboptimal but usable. **Of those 32**, 19 cost less than twenty per cent and the median costs 16.9 per cent; over all 38, including the six failures, the median is unbounded above the twentieth percentile of cost. The denominators are stated because the two readings differ and the paper is about that difference. Accuracy compresses that spread into a single proportion and reports the twenty-per-cent miss and the non-converging pick as the same event. On F6 the selector is wrong on fifty-eight per cent of operators at a median cost, over the converging misses alone, of twenty-five per cent; on F4 it is wrong on seventy-five per cent at a median cost of five hundred and ninety-six per cent, and that figure omits five operators where the cost was unbounded. The accuracy column does not separate them.

8.4 Failure modes

Every non-converging selection in this paper is the same event, and naming it avoids an ambiguity. All six exhausted the declared iteration cap of twenty thousand without reaching a true relative residual of 10^{-8} . None hit the thirty-second wall-clock cap and none broke down on non-positive curvature, though both outcomes are recorded separately in the raw file and nineteen runs across the whole sweep did time out. Where this paper writes that a choice did not converge, it means the iteration cap.

8.5 The abstention rule does not work

The pre-registered rule refuses when the test operator’s feature vector lies beyond the ninety-ninth percentile of the calibration split’s Mahalanobis distances. The declared fallback, fixed in the pre-registration, is the method with the best median rank on the training partition of that fold, which is Jacobi on five folds and no preconditioner on one. Under leave-one-family-out the rule refuses 78.5 per cent of held-out operators, which is the correct answer in the narrow sense that a new family is outside the training support, and a useless one in practice.

The rule does not buy what it was built to buy. Pooled ninetieth-percentile regret with abstention and the fallback is 0.347, against 0.352 without, and the median improves from 0.10 to 0.00 because the fallback is a reasonable default on the easy folds. Against that, the fallback *doubles* the number of operators on which the deployed choice never converges, from six to twelve. Those twelve are excluded from the percentile, so the two tail figures are not computed over the same set and the near-equality of 0.347 and 0.352 overstates how little changed. On F4, where refusing would have mattered most, coverage is 0.33 and the finite tail falls from 7.96 to 1.73 while four operators still carry unbounded regret.

Before drawing a conclusion from that, one objection has to be closed. Three of the twelve features are linearly dependent by construction, so the calibration covariance is singular to machine precision and the ridge of 10^{-6} used to invert it leaves a condition number near 10^8 . A distance dominated by an arbitrary direction would refuse nearly everything for a reason that has nothing to do with the operators. We tested it. Dropping the dependent feature leaves pooled coverage unchanged at 0.215; replacing the ridged inverse with a pseudo-inverse lowers

coverage to 0.092, refusing more. The refusal rate is a property of the held-out families, not of the conditioning.

The result is therefore sharper than *the rule is badly tuned*. Distance from the training features in the space the selector was fitted on does not identify the operators whose *decision* is unlike the training set. F4 is not merely far away: it contains a qualitative change in which methods converge at all, and a geometric test on the feature space cannot see a change in the decision surface. We report this as a failed design.

8.6 No preconditioner at all

On 23 of the 65 operators the fastest method is unpreconditioned conjugate gradients. On F3 it is the oracle on all twelve, and on F6 on eleven of twelve. Once setup cost and per-iteration cost are counted end to end, a preconditioner that reduces the iteration count can still lose. A selector whose candidate set contains only preconditioners would be wrong on a third of this grid by construction, and the literature convention of comparing preconditioners to each other rather than to nothing hides that.

8.7 The probe is a net loss on cheap problems

Probe cost runs from 0.027 to 0.485 seconds, with a median of 0.056 seconds. Against the fastest converging solve on the same operator the median ratio is 0.24, and on 3 of 65 operators the probe costs more than the whole solve it is meant to inform, by up to a factor of 6.9. Measured against solve time with setup and probe removed, which is the narrower reading, the median ratio is 0.31 and the probe exceeds the solve on 9 of 65 by up to 12.3. The first pair is the one the regret definition of §7 implies, because T there is end to end. The crossover is not exotic: it is simply that a probe with a fixed budget is amortised over a solve whose cost varies by four orders of magnitude across this grid. Any deployment that charges the probe per linear system, rather than amortising it across a sweep of right-hand sides or a family of related operators, should measure this ratio first.

This has a consequence for the score itself, and it is the one post-hoc analysis in the paper. The registered regret of §7 charges the probe to a method that needs it, which is the Chebyshev preconditioner alone. That is the right accounting for a method and the wrong accounting for a selector, because the selector consumes the probe on every operator in order to choose at all, whatever it goes on to pick. Charging it once to every choice, with no second charge when the choice is Chebyshev and none against the clairvoyant oracle, median regret moves from 0.10 to 0.41 and the ninetieth percentile from 0.35 to 3.40. The non-converging count is unchanged at six, because paying for a probe cannot make a divergent solve converge. Per family the effect tracks how cheap the solves are: on F6, where unpreconditioned conjugate gradients is the oracle on eleven of twelve operators and those solves are fast, the tail goes from 0.27 to 3.21. A selector can be right about the ranking and still lose, because the act of deciding cost more than the decision saved. We report it beside the registered number.

8.8 Recurrence residual

On 25 of the 432 runs the recurrence residual carried inside conjugate gradients fell below the tolerance while the true relative residual did not. On a two-dimensional conduction operator at contrast 10^5 four methods reported a relative residual of 10^{-8} with true values from 8.3×10^{-8}

to 1.8×10^{-7} . A harness that trusted the recurrence, as harnesses commonly do because recomputing the true residual costs an extra operator application, would have recorded those runs as converged and credited the methods with a speedup they did not achieve. The phenomenon is well understood: the attainable accuracy of a recursively computed residual was characterised by Greenbaum [4], and residual replacement strategies exist for it [5]. We add the rate on this grid and the observation below about what it does to selection labels. We recompute the true residual every hundred iterations and at exit, at an overhead of about one per cent, and we recommend that any timing study of iterative solvers state which residual its convergence test used.

This is not a detail orthogonal to the rest of the paper. The oracle for every operator here is the fastest method that converged, and every regret in this paper is measured against that label. A harness that trusted the recurrence would have promoted some unconverged runs to oracle, and a selector scored against those labels would have been rewarded for reproducing them. A selector is only as good as the oracle it is scored against.

9 What this study does not show

The operators are generated, not drawn from a public collection of application matrices. The machine used for the sweep sits behind an egress policy that blocks the SuiteSparse and Matrix Market hosts, so we generated the grid from declared discretisations and left the restriction in place, and the reader should treat every number here as conditional on that choice. Generated operators understate the messiness of assembled application matrices and probably understate the difficulty of selection. All timings are single-node and single-threaded in one process; nothing here speaks to distributed solves, where the communication term changes the ranking. The candidate set is six classical methods and contains no multigrid, no domain decomposition and no learned or discovered solver, so no claim in this paper compares against the state of the art in preconditioning. The learner is one gradient-boosted regressor per method with fixed hyperparameters and we did not tune it. What tuning would do to F4 we do not know: we did not run that experiment, and an untested prediction about it has no place here. Sixty-five usable operators is a small sample and the per-fold columns of Table 3 rest on six to twelve operators each, so they are counts.

10 The reproduction package

Everything this paper rests on is attached to it as ancillary files, so there is no repository to go stale and no link to rot. Under `anc/` are the pre-registration with its three addenda, the operator generators, the solvers, the probe, the sweep runner, the scoring script, the raw result file, the scores, the run metadata and two SHA-256 manifests. The raw file carries one JSON record per operator and method, including the runs that failed; the scoring script reads only that file. Running `run_sweep.py` then `score.py` reproduces every number in this paper from scratch in about half an hour on one core, with no network access, and `shasum -a 256 -c ARTIFACTS.sha256` checks the set first. `score.py` records the interpreter, numpy, scipy and scikit-learn versions into `scores.json` and emits the probe figures of §8.7 and the miss statistics of §8.3, including the per-fold counts behind the F4 and F6 figures, and `requirements.txt` pins the stack that produced the first row of Table 2. A different stack will reproduce every number here except the median and the accuracy, for the reason given in §8.1. Two of the three addenda

are post-hoc and say so; `PREREG.sha256` carries the hash and timestamp at which each was locked, so the order of events is checkable.

11 Conclusion

The selector we built fails the tail and accuracy criteria we registered in advance wherever we run it, fails the median criterion on three machines of five, and the catastrophic picks fall in one family out of six, though four folds miss the registered bars. In that family the correct choice stops being a matter of speed and starts being a matter of whether the solve finishes, and no amount of accuracy on the other five predicts it. The abstention rule that was supposed to catch this case refused most held-out operators without protecting the tail.

The useful reading is not that spectral-probe selection contains no signal. It separates clearly from the shuffled-label control in the tail and it generalises well on several held-out families, perfectly on one. The failure is that the same representation does not protect the tail when the unseen family changes the structure of the decision, and that neither the median nor a distance in feature space reveals that it has. The reported quantity has to change: hold out families, score in regret, publish the tail beside the count of picks that did not converge, keep *use none* in the candidate set, and charge the probe against the solve it was acquired to inform.

References

- [1] E. Jessup, P. Motter, B. Norris and K. Sood, Performance-based numerical solver selection in the Lighthouse framework, *SIAM Journal on Scientific Computing* 38(5), S750–S771, 2016.
- [2] H. Xiong and others, MM-AutoSolver: a multimodal machine learning method for the auto-selection of iterative solvers and preconditioners, *Journal of Parallel and Distributed Computing*, 2025.
- [3] B. Bischl, P. Kerschke, L. Kotthoff, M. Lindauer, Y. Malitsky, A. Fréchette, H. H. Hoos, F. Hutter, K. Leyton-Brown, K. Tierney and J. Vanschoren, ASlib: a benchmark library for algorithm selection, *Artificial Intelligence* 237, 41–58, 2016.
- [4] A. Greenbaum, Estimating the attainable accuracy of recursively computed residual methods, *SIAM Journal on Matrix Analysis and Applications*, 1997, doi:10.1137/S0895479895284944.
- [5] H. A. van der Vorst and Q. Ye, Residual replacement strategies for Krylov subspace iterative methods for the convergence of true residuals, *SIAM Journal on Scientific Computing*, 1999, doi:10.1137/S1064827599353865.